

1.开发前必读

1.1 控制器型号选择

本说明只适用于BX系列五代六代控制卡（不包括字库卡），例如：BX-5E1、BX-5M1、BX-6E1X等，其中BX-5E1、BX-5E2、BX-5E3支持动态区（4个），BX-6E1、BX-6E2、BX-6E1X、BX-6E2X、BX-6Q系列支持动态区（32个）。

1.2 关于动态区

根据存储介质的不同，我们将控制器上现实的内容分为两类：普通节目（数据存储在flash中）和动态区（数据存储在ram中）。普通节目可以包括各种区域：时间区、传感器区、图文区等。

普通节目，会被存储在控制器的flash中，其内容掉电不丢失，但是因为flash存储器擦写寿命只有十万次，所以，其不能用于更新频率很高的场合，例如：停车场车位信息，外部传感器状态的实时更新，车次状态的实时更新等。

动态区完全独立于普通节目，其内容可以按区域进行单独更新，它可以与普通节目一起播放，也可以单独播放，它最大的特点是内容存储在ram中，没有擦写次数限制，内容掉电不保存，通常用于信息更新频繁场合。

1.3 准备工作

在进行开发之前，请使用通用软件LedshowTW图文编辑软件对控制器进行相关配置，比如控制器IP地址（串口号）、屏幕参数、扫描方式等，在使用LedshowTW软件能发送普通节目到控制器并能显示正常后，在进行二次开发。

1.4 常见问题

- 关于表格 控制器不支持Excel或其他表格，所以，需要将表格转换成图片添加到区域中

2. 接口说明【5代和6代接口函数名称不一样，本文主要以6代控制卡接口函数为主】

2.1 SDK初始化

```
/// <summary>
/// windows平台需要初始化SDK（linux会自动初始化）
/// </summary>
/// <returns></returns>
[DllImport("bx_sdk_dual.dll", CharSet = CharSet.Unicode)]
public static extern int bxDual_InitSdk();
```

2.2 节目编辑

//通过接口函数判定接口函数是否成功，0成功，-1为通讯失败，其它值为其它错误

//指定IP ping控制卡获取控制卡数据，屏参相关参数已知情况可省略该步骤

```
bxDualSdk.Ping_data data = new bxDualSdk.Ping_data();  
int err = bxDualCmdTcpPing(Program.ip, Program.port, ref data);
```

//显示屏屏基色

```
byte cmb_ping_Color = 1;
```

```
if (data.Color == 1) { cmb_ping_Color = 1; } //单基色
```

```
else if (data.Color == 3) { cmb_ping_Color = 2; } //双基色
```

```
else if (data.Color == 7) { cmb_ping_Color = 3; } //三基色【无灰度全彩七
```

色】

```
else { cmb_ping_Color = 4; } //全彩色
```

//设置屏幕参数相关，参数不对会导致发送的节目内容出现黑屏，乱码，杂点多种情况，以现场硬件规格参数为准 发送节目必要接口，发送动态区可忽略

```
err =
```

```
bxDualProgramSetScreenParams_G56((bxDualSdk.E_ScreenColor_G56)cmb_ping_Color,  
data.ControllerType, bxDualSdk.E_DoubleColorPixel_G56.eDOUBLE_COLOR_PIXTYPE_1);
```

//创建节目，设置节目属性，可以进行节目时长或者有效播放播放日期和时间进行设置，一次只能编辑一个节目下发，根据ProgramID区分节目，相同ID的节目会替换，不同ID的节目默认轮流循环播放

```
bxDualSdk.EQProgramHeader_G6 header;  
header.FileType = 0x00;  
header.ProgramID = 0;  
header.ProgramStyle = 0x00;  
header.ProgramPriority = 0x00;  
header.ProgramPlayTimes = 1;  
header.ProgramTimeSpan = 0;  
header.SpecialFlag = 0;  
header.CommExtendParaLen = 0x00;  
header.ScheduNum = 0;  
header.LoopValue = 0;  
header.Intergrate = 0x00;  
header.TimeAttributeNum = 0x00;  
header.TimeAttribute0Offset = 0x0000;  
header.ProgramWeek = 0xff;  
header.ProgramLifeSpan_sy = 0xffff;  
header.ProgramLifeSpan_sm = 0x03;  
header.ProgramLifeSpan_sd = 0x14;  
header.ProgramLifeSpan_ey = 0xffff;  
header.ProgramLifeSpan_em = 0x03;  
header.ProgramLifeSpan_ed = 0x14;  
header.PlayPeriodGrpNum = 0;  
err = bxDualSdk.bxDualProgramAddProgram_G6(ref header);
```

//创建显示分区，设置区域显示位置，示例创建一个区域编号为0，区域大小64*32的图文

分区

```
bxDualSdk.EQAreaHeader_G6 aheader;  
aheader.AreaType = 0;  
aheader.AreaX = 0;  
aheader.AreaY = 0;  
aheader.AreaWidth = 128;  
aheader.AreaHeight = 32;  
aheader.BackGroundFlag = 0x00;  
aheader.Transparency = 101;  
aheader.AreaEqual = 0x00;  
bxDualSdk.EQSound_6G stSoundData = new bxDualSdk.EQSound_6G();
```

区域

```
stSoundData.SoundFlag = 0;
stSoundData.SoundVolum = 0;
stSoundData.SoundSpeed = 0;
stSoundData.SoundDataMode = 0;
stSoundData.SoundReplayTimes = 0;
stSoundData.SoundReplayDelay = 0;
stSoundData.SoundReservedParaLen = 0;
stSoundData.Soundnumdeaf = 0;
stSoundData.Soundlanguages = 0;
stSoundData.Soundwordstyle = 0;
stSoundData.SoundDataLen = 0;
byte[] t = new byte[1];
t[0] = 0;
stSoundData.SoundData = IntPtr.Zero;
aheader.stSoundData = stSoundData;
err = bxdualsdk.bxDual_program_addArea_G6(0, ref aheader); //添加图文

Console.WriteLine("bxDual_program_addArea_G6:" + err);

//添加显示内容，此处为图文分区0添加字符串
byte[] Font = Encoding.GetEncoding("GBK").GetBytes("宋体");
IntPtr font = Marshal.AllocHGlobal(Font.Length);
Marshal.Copy(Font, 0, font, Font.Length);
byte[] strAreaTxtContent =
Encoding.GetEncoding("GBK").GetBytes("111111\0");
IntPtr str = Marshal.AllocHGlobal(strAreaTxtContent.Length);
Marshal.Copy(strAreaTxtContent, 0, str, strAreaTxtContent.Length);
bxdualsdk.EQpageHeader_G6 pheader;
pheader.PageStyle = 0x00;
pheader.DisplayMode = 0x04; //移动模式
pheader.ClearMode = 0x01;
pheader.Speed = 15; //速度
pheader.StayTime = 0; //停留时间
pheader.RepeatTime = 1;
pheader.ValidLen = 10;
pheader.CartoonFrameRate = 0x00;
pheader.BackNotValidFlag = 0x00;
pheader.arrMode = bxdualsdk.E_arrMode.eSINGLELINE;
pheader.fontSize = 10;
pheader.color = (uint)0x01;
pheader.fontBold = 0;
pheader.fontItalic = 0;
pheader.tdirection = bxdualsdk.E_txtDirection.pNORMAL;
pheader.txtSpace = 0;
pheader.Valign = 1;
pheader.Halign = 0;
err = bxdualsdk.bxDual_program_picturesAreaAddTxt_G6(0,
strAreaTxtContent, Font, ref pheader);
Console.WriteLine("bxDual_program_picturesAreaAddTxt_G6:" + err);

//创建显示分区，设置区域显示位置，示例创建一个区域编号为1，区域大小64 * 32的时间分区，Y轴64，区域之间不可重叠
bxdualsdk.EQareaHeader_G6 aheader1;
aheader1.AreaType = 2;
aheader1.AreaX = 0;
aheader1.AreaY = 32;
aheader1.Areawidth = 128;
aheader1.AreaHeight = 32;
```

```

aheader1.BackGroundFlag = 0x00;
aheader1.Transparency = 101;
aheader1.AreaEqual = 0x00;
aheader1.stSoundData = stSoundData;
err = bxdualsdk.bxDual_program_addArea_G6(1, ref aheader1);
Console.WriteLine("bxDual_program_addArea_G6:" + err);

//添加时间区域显示内容
bxdualsdk.EQtimeAreaData_G56 timeData2;
timeData2.linestyle = bxdualsdk.E_arrMode.eMULTILINE;
timeData2.color = (uint)bxdualsdk.E_Color_G56.eRED;
timeData2.fontName = "simsun";
timeData2.fontSize = 10;
timeData2.fontBold = 0;
timeData2.fontItalic = 0;
timeData2.fontUnderline = 0;
timeData2.fontAlign = 1; //0--左对齐, 1-居中, 2-右对齐
timeData2.date_enable = 0;
timeData2.datestyle = bxdualsdk.E_DateStyle.eYYYY_MM_DD_MINUS;
timeData2.time_enable = 1;
timeData2.timestyle = bxdualsdk.E_TimeStyle.eHH_MM_COLON;
timeData2.week_enable = 0;
timeData2.weekstyle = bxdualsdk.E_WeekStyle.eMonday;
err = bxdualsdk.bxDual_program_timeAreaAddContent_G6(1, ref
timeData2);
Console.WriteLine("bxDual_program_timeAreaAddContent_G6:" + err);

//发送节目到显示屏
bxdualsdk.EQprogram_G6 program = new bxdualsdk.EQprogram_G6();
err = bxdualsdk.bxDual_program_IntegrateProgramFile_G6(ref program);
Console.WriteLine("bxDual_program_IntegrateProgramFile_G6:" + err);
err = bxdualsdk.bxDual_program_deleteProgram_G6();
Console.WriteLine("bxDual_program_deleteProgram_G6:" + err);

if (Program.check)//网口
{
    err = bxdualsdk.bxDual_cmd_ofsStartFileTransf(Program.ip,
Program.port);
    Console.WriteLine("bxDual_cmd_ofsStartFileTransf:" + err);

    err = bxdualsdk.bxDual_cmd_ofsWriteFile(Program.ip, Program.port,
program.dfileName, program.dfileType, program.dfileLen, 1, program.dfileAddre);
    Console.WriteLine("bxDual_cmd_ofsWriteFile:" + err);
    err = bxdualsdk.bxDual_cmd_ofsWriteFile(Program.ip, Program.port,
program.fileName, program.fileType, program.fileLen, 1, program.fileAddre);
    Console.WriteLine("bxDual_cmd_ofsWriteFile:" + err);
    err = bxdualsdk.bxDual_cmd_ofsEndFileTransf(Program.ip,
Program.port);
    Console.WriteLine("bxDual_cmd_ofsEndFileTransf:" + err);
}
else//串口
{
    err = bxdualsdk.bxDual_cmd_uart_ofsStartFileTransf(Program.com,
Program.baudRate);
    Console.WriteLine("bxDual_cmd_uart_ofsStartFileTransf:" + err);
}

```

```

        err = bxdualsdk.bxDual_cmd_uart_ofsWriteFile(Program.com,
Program.baudRate, program.dfileName, program.dfileType, program.dfileLen, 1,
program.dfileAddre);
        Console.WriteLine("bxDual_cmd_uart_ofsWriteFile:" + err);
        err = bxdualsdk.bxDual_cmd_uart_ofsWriteFile(Program.com,
Program.baudRate, program.fileName, program.fileType, program.fileLen, 1,
program.fileAddre);
        Console.WriteLine("bxDual_cmd_uart_ofsWriteFile:" + err);
        err = bxdualsdk.bxDual_cmd_uart_ofsEndFileTransf(Program.com,
Program.baudRate);
    }

    err = bxdualsdk.bxDual_program_freeBuffer_G6(ref program);
    Console.WriteLine("bxDual_program_freeBuffer_G6:" + err);

```

2.3 动态区编辑【仅BX-5E系列，BX-6E系列，BX-6Q系列支持】

```

bxdualsdk.EQareaHeader_G6 aheader;
aheader.AreaType = 0x10;
aheader.AreaX = 0;
aheader.AreaY = 0;
aheader.AreaWidth = 64;
aheader.AreaHeight = 32;
aheader.BackGroundFlag = 0x00;
aheader.Transparency = 101;
aheader.AreaEqual = 0x00;
bxdualsdk.EQSound_6G stSoundData = new bxdualsdk.EQSound_6G();
byte[] strSoundTxt = Encoding.GetEncoding("GB2312").GetBytes("插入
ab34测试语音");
stSoundData.SoundFlag = 0x00;
stSoundData.SoundPerson = 0x01;
stSoundData.SoundVolum = 2;
stSoundData.SoundSpeed = 0x2;
stSoundData.SoundDataMode = 0x00;
stSoundData.SoundReplayTimes = 0x01;
stSoundData.SoundReplayDelay = 200;
stSoundData.SoundReservedParaLen = 0x03;
stSoundData.Soundnumdeal = 0x00;
stSoundData.Soundlanguages = 0x00;
stSoundData.Soundwordstyle = 0x00;
stSoundData.SoundDataLen = strSoundTxt.Length;
stSoundData.SoundData = Class1.BytesToIntptr(strSoundTxt);
aheader.stSoundData = stSoundData;
Random ran = new Random();
bxdualsdk.EQpageHeader_G6 pheader;
pheader.PageStyle = 0x00;
pheader.DisplayMode = 0;
pheader.ClearMode = 0x01;
pheader.Speed = 10;
pheader.StayTime = 100;
pheader.RepeatTime = 1;
pheader.ValidLen = 64;
pheader.CartoonFrameRate = 0x00;

```

```

pheader.BackNotValidFlag = 0x00;
pheader.arrMode = bxdualsdk.E_arrMode.eMULTILINE;
pheader.fontSize = 12;
pheader.color = (uint)1;
pheader.fontBold = 0;
pheader.fontItalic = 0;
pheader.tdirection = bxdualsdk.E_txtDirection.pNORMAL;
pheader.txtSpace = 5;
pheader.Valign = 2;
pheader.Halign = 1;
byte[] Font = Encoding.GetEncoding("GBK").GetBytes("宋体\0");
fontName = Marshal.AllocHGlobal(Font.Length);
Marshal.Copy(Font, 0, fontName, Font.Length);
byte[] str =
Encoding.GetEncoding("GBK").GetBytes("123\n4567899999999999\0");
strAreaTxtContent = Marshal.AllocHGlobal(str.Length);
Marshal.Copy(str, 0, strAreaTxtContent, str.Length);
Console.WriteLine("发送节目完成" + DateTime.Now.ToString());

//设置屏幕点阵类型，非必须
//bxdualsdk.bxDual_dynamicArea_SetDualPixel(bxdualsdk.E_DoubleColorPixel_G56.eDOUBLE_COLOR_PIXTYPE_2);
//动态区优先播放，节目停止播放
err =
bxdualsdk.bxDual_dynamicArea_AddAreaTxtDetails_6G(Program.ip, Program.port,
bxdualsdk.E_ScreenColor_G56.eSCREEN_COLOR_DOUBLE, AreaId,
ref aheader, ref pheader, fontName, strAreaTxtContent);
//是否关关节目，和节目一起播放，显示位置和节目下的区域不能有重叠
//err =
bxdualsdk.bxDual_dynamicArea_AddAreaTxtDetails_withProgram_6G(Program.ip,
Program.port, bxdualsdk.E_ScreenColor_G56.eSCREEN_COLOR_DOUBLE, AreaId, ref
aheader, ref pheader, fontName, strAreaTxtContent, RelateProNum,
RelateProSerial);

```

2.4 其它设置

2.4.1 校时命令

```

/*! *****
* 函数名:      cmd_check_time ()
* 参数名: ip: 控制器IP, port: 控制器端口
* 返回值: 0 成功, 其他值为错误号
* 功 能: 校时, 让控制器和当前上位机所在系统时间一致
* 注:
*
*****/
[DllImport("bx_sdk_dual.dll", CharSet = CharSet.Unicode)]
public static extern int bxDual_cmd_check_time(byte[] ip, ushort port);

```

2.4.2 开关机

```

//网口
    err = bxdualsdk.bxDual_cmd_coerceOnOff(Program.ip, Program.port,
0); //关机
    err = bxdualsdk.bxDual_cmd_coerceOnOff(Program.ip, Program.port,
1); //开机
//串口
    err = bxdualsdk.bxDual_cmd_coerceOnOff_uart(Program.com,
Program.baudRate, 0); //关机
    err = bxdualsdk.bxDual_cmd_coerceOnOff_uart(Program.com,
Program.baudRate, 1); //开机

```

2.4.3 亮度设置

```

bxdualsdk.Brightness brightness;
brightness.BrightnessMode = 0;
brightness.HalfHourValue0 = num;
brightness.HalfHourValue1 = num;
brightness.HalfHourValue2 = num;
brightness.HalfHourValue3 = num;
brightness.HalfHourValue4 = num;
brightness.HalfHourValue5 = num;
brightness.HalfHourValue6 = num;
brightness.HalfHourValue7 = num;
brightness.HalfHourValue8 = num;
brightness.HalfHourValue9 = num;
brightness.HalfHourValue10 = num;
brightness.HalfHourValue11 = num;
brightness.HalfHourValue12 = num;
brightness.HalfHourValue13 = num;
brightness.HalfHourValue14 = num;
brightness.HalfHourValue15 = num;
brightness.HalfHourValue16 = num;
brightness.HalfHourValue17 = num;
brightness.HalfHourValue18 = num;
brightness.HalfHourValue19 = num;
brightness.HalfHourValue20 = num;
brightness.HalfHourValue21 = num;
brightness.HalfHourValue22 = num;
brightness.HalfHourValue23 = num;
brightness.HalfHourValue24 = num;
brightness.HalfHourValue25 = num;
brightness.HalfHourValue26 = num;
brightness.HalfHourValue27 = num;
brightness.HalfHourValue28 = num;
brightness.HalfHourValue29 = num;
brightness.HalfHourValue30 = num;
brightness.HalfHourValue31 = num;
brightness.HalfHourValue32 = num;
brightness.HalfHourValue33 = num;
brightness.HalfHourValue34 = num;
brightness.HalfHourValue35 = num;
brightness.HalfHourValue36 = num;
brightness.HalfHourValue37 = num;
brightness.HalfHourValue38 = num;

```

```

        brightness.HalfHourValue39 = num;
        brightness.HalfHourValue40 = num;
        brightness.HalfHourValue41 = num;
        brightness.HalfHourValue42 = num;
        brightness.HalfHourValue43 = num;
        brightness.HalfHourValue44 = num;
        brightness.HalfHourValue45 = num;
        brightness.HalfHourValue46 = num;
        brightness.HalfHourValue47 = num;

        //网口
        err = bxdualsdk.bxDual_cmd_setBrightness(Program.ip,
Program.port, ref brightness);
        //串口
        err = bxdualsdk.bxDual_cmd_setBrightness_uart(Program.com,
Program.baudRate, ref brightness);

```

2.5 控制卡型号表

```

// 控制器类型
private const ushort BX_5AT = 0x0051;
private const ushort BX_5A0 = 0x0151;
private const ushort BX_5A1 = 0x0251;
private const ushort BX_5A2 = 0x0351;
private const ushort BX_5A3 = 0x0451;
private const ushort BX_5A4 = 0x0551;
private const ushort BX_5A1_WIFI = 0x0651;
private const ushort BX_5A2_WIFI = 0x0751;
private const ushort BX_5A4_WIFI = 0x0851;
private const ushort BX_5A = 0x0951;
private const ushort BX_5A2_RF = 0x1351;
private const ushort BX_5A4_RF = 0x1551;
private const ushort BX_5AT_WIFI = 0x1651;
private const ushort BX_5AL = 0x1851;

private const ushort AX_AT = 0x2051;
private const ushort AX_A0 = 0x2151;

private const ushort BX_5MT = 0x0552;
private const ushort BX_5M1 = 0x0052;
private const ushort BX_5M1X = 0x0152;
private const ushort BX_5M2 = 0x0252;
private const ushort BX_5M3 = 0x0352;
private const ushort BX_5M4 = 0x0452;

private const ushort BX_5E1 = 0x0154;
private const ushort BX_5E2 = 0x0254;
private const ushort BX_5E3 = 0x0354;

private const ushort BX_5UT = 0x0055;
private const ushort BX_5U0 = 0x0155;
private const ushort BX_5U1 = 0x0255;
private const ushort BX_5U2 = 0x0355;
private const ushort BX_5U3 = 0x0455;

```



```
private const ushort BX_5U4 = 0x0555;
private const ushort BX_5U5 = 0x0655;
private const ushort BX_5U = 0x0755;
private const ushort BX_5UL = 0x0855;

private const ushort AX_UL = 0x2055;
private const ushort AX_UT = 0x2155;
private const ushort AX_U0 = 0x2255;
private const ushort AX_U1 = 0x2355;
private const ushort AX_U2 = 0x2455;

private const ushort BX_5Q0 = 0x0056;
private const ushort BX_5Q1 = 0x0156;
private const ushort BX_5Q2 = 0x0256;
private const ushort BX_5Q0P = 0x1056;
private const ushort BX_5Q1P = 0x1156;
private const ushort BX_5Q2P = 0x1256;
private const ushort BX_5QL = 0x1356;

private const ushort BX_5QS1 = 0x0157;
private const ushort BX_5QS2 = 0x0257;
private const ushort BX_5QS = 0x0357;
private const ushort BX_5QS1P = 0x1157;
private const ushort BX_5QS2P = 0x1257;
private const ushort BX_5QSP = 0x1357;

private const ushort BX_6M0 = 0x0062;
private const ushort BX_6M1 = 0x0162;
private const ushort BX_6M2 = 0x0262;
private const ushort BX_6M3 = 0x0362;
private const ushort BX_6M = 0x0462;
private const ushort BX_6MT = 0x0562;
private const ushort BX_6M2JT = 0x3262;
private const ushort BX_6M0P = 0x0062;
private const ushort BX_6M1P = 0x0162;
private const ushort BX_6M2P = 0x4262;
private const ushort BX_6M3P = 0x4362;
private const ushort BX_6M4P = 0x4462;
private const ushort JT1 = 0x5162;
private const ushort JT2 = 0x5262;
private const ushort JT3 = 0x5362;

private const ushort BX_6M0_YY = 0x1062;
private const ushort BX_6M1_YY = 0x1162;
private const ushort BX_6M2_YY = 0x1262;
private const ushort BX_6M3_YY = 0x1362;
private const ushort BX_6M_YY = 0x1462;

private const ushort BX_6X1 = 0x2162;
private const ushort BX_6X2 = 0x2262;
private const ushort BX_6X3 = 0x2362;

private const ushort BX_6U0 = 0x0063;
private const ushort BX_6U1 = 0x0163;
private const ushort BX_6U2 = 0x0263;
private const ushort BX_6U3 = 0x0363;
private const ushort BX_6U = 0x0463;
private const ushort BX_6UT = 0x0563;
```

```

private const ushort BX_6U0_YY = 0x1063;
private const ushort BX_6U1_YY = 0x1163;
private const ushort BX_6U2_YY = 0x1263;
private const ushort BX_6U3_YY = 0x1363;
private const ushort BX_6U_YY = 0x1463;

private const ushort BX_6A0 = 0x2063;
private const ushort BX_6A1 = 0x2163;
private const ushort BX_6A2 = 0x2263;
private const ushort BX_6A3 = 0x2363;
private const ushort BX_6A = 0x2463;

private const ushort BX_6A0_YY = 0x3063;
private const ushort BX_6A1_YY = 0x3163;
private const ushort BX_6A2_YY = 0x3263;
private const ushort BX_6A3_YY = 0x3363;
private const ushort BX_6A_YY = 0x3463;

private const ushort BX_6A0_G = 0x4063;
private const ushort BX_6A1_G = 0x4163;
private const ushort BX_6A2_G = 0x4263;
private const ushort BX_6A3_G = 0x4363;
private const ushort BX_6AT_G = 0x4463;

private const ushort BX_6S1 = 0x5163;
private const ushort BX_6S2 = 0x5263;
private const ushort BX_6S3 = 0x5363;

private const ushort BX_6W0 = 0x0064;
private const ushort BX_6W1 = 0x0164;
private const ushort BX_6W2 = 0x0264;
private const ushort BX_6W3 = 0x0364;
private const ushort BX_6W = 0x0464;
private const ushort BX_6WT = 0x0564;

private const ushort BX_6E1 = 0x0174;
private const ushort BX_6E2 = 0x0274;
private const ushort BX_6E3 = 0x0374;
private const ushort BX_6E1X = 0x0474;
private const ushort BX_6E2X = 0x0574;
private const ushort BX_6E1XP = 0x0674;
private const ushort BX_6E2XP = 0x0774;
private const ushort BX_6E3P = 0x0974;

private const ushort BX_6Q0 = 0x0066;
private const ushort BX_6Q1 = 0x0166;
private const ushort BX_6Q2 = 0x0266;
private const ushort BX_6Q2L = 0x0466;
private const ushort BX_6Q3 = 0x0366;
private const ushort BX_6Q3L = 0x0566;

```

2.6 错误码

```

public class sdk_err
{

```

```

    public const int ERR_NO = 0;
    public const int ERR_OUTOFGROUP = 1;
    public const int ERR_NOCMD = 2;
    public const int ERR_BUSY = 3;
    public const int ERR_MEMORYVOLUME = 4;
    public const int ERR_CHECKSUM = 5;
    public const int ERR_FILENOTEXIST = 6;
    public const int ERR_FLASH = 7;
    public const int ERR_FILE_DOWNLOAD = 8;
    public const int ERR_FILE_NAME = 9;
    public const int ERR_FILE_TYPE = 10;
    public const int ERR_FILE_CRC16 = 11;
    public const int ERR_FONT_NOT_EXIST = 12;
    public const int ERR_FIRMWARE_TYPE = 13;
    public const int ERR_DATE_TIME_FORMAT = 14;
    public const int ERR_FILE_EXIST = 15;
    public const int ERR_FILE_BLOCK_NUM = 16;
    public const int ERR_CONTROLLER_TYPE = 17;
    public const int ERR_SCREEN_PARA = 18;
    public const int ERR_CONTROLLER_ID = 19;
    public const int ERR_USER_SECRET = 20;
    public const int ERR_OLD_USER_SECRET = 21;
    public const int ERR_PHY1_NO_SECRET = 22;
    public const int ERR_PHY1_USE_SECRET = 23;
    public const int ERR_RTC = 24;
    public const int ERR_CMD_PARA = 25;
    public const int ERR_CASCADE_COMM = 26;
    public const int ERR_NO_BATTLE_AREA = 27;
    public const int ERR_NO_TIMER_AREA = 28;
    public const int ERR_FPGA_COMM = 29;
    public const int ERR_SET_MODBUS_PARA = 60;

    public const int ERR_TCP = -1;
}

public static string GetError(int err)
{
    string str = "";
    switch (err)
    {
        case sdk_err.ERR_TCP:
            str = "通讯错误";
            break;
        case sdk_err.ERR_NO:
            str = "没有错误";
            break;
        case sdk_err.ERR_OUTOFGROUP:
            str = "命令组错误";
            break;
        case sdk_err.ERR_NOCMD:
            str = "此命令不存在";
            break;
        case sdk_err.ERR_BUSY:
            str = "控制器忙";
            break;
        case sdk_err.ERR_MEMORYVOLUME:
            str = "存储器容量越界";
            break;
        case sdk_err.ERR_CHECKSUM:

```

```
        str = "数据包 CRC 校验错误";
        break;
    case sdk_err.ERR_FILENOTEXIST:
        str = "此文件不存在";
        break;
    case sdk_err.ERR_FLASH:
        str = "Flash 访问错误";
        break;
    case sdk_err.ERR_FILE_DOWNLOAD:
        str = "文件下载错误";
        break;
    case sdk_err.ERR_FILE_NAME:
        str = "文件名错误";
        break;
    case sdk_err.ERR_FILE_TYPE:
        str = "文件类型错误";
        break;
    case sdk_err.ERR_FILE_CRC16:
        str = "文件校验错误";
        break;
    case sdk_err.ERR_FONT_NOT_EXIST:
        str = "字库文件不存在";
        break;
    case sdk_err.ERR_FIRMWARE_TYPE:
        str = "Firmware 与控制器类型不匹配";
        break;
    case sdk_err.ERR_DATE_TIME_FORMAT:
        str = "日期时间格式错误 ";
        break;
    case sdk_err.ERR_FILE_EXIST:
        str = "此文件已存在";
        break;
    case sdk_err.ERR_FILE_BLOCK_NUM:
        str = "文件 block 号错误";
        break;
    case sdk_err.ERR_CONTROLLER_TYPE:
        str = "控制器类型不匹配";
        break;
    case sdk_err.ERR_SCREEN_PARA:
        str = "控制器参数越界或错误";
        break;
    case sdk_err.ERR_CONTROLLER_ID:
        str = "控制器 ID 错误";
        break;
    case sdk_err.ERR_USER_SECRET:
        str = "用户密码错误";
        break;
    case sdk_err.ERR_OLD_USER_SECRET:
        str = "设置新密码时，输入的旧密码不正确";
        break;
    case sdk_err.ERR_PHY1_NO_SECRET:
        str = "底层无密码，上位机有密码";
        break;
    case sdk_err.ERR_PHY1_USE_SECRET:
        str = " 底层有密码，上位机无密码";
        break;
    case sdk_err.ERR_RTC:
        str = "时钟芯片故障";
```

```
        break;
    case sdk_err.ERR_CMD_PARA:
        str = "命令参数错误";
        break;
    case sdk_err.ERR_CASCADE_COMM:
        str = "级联系统通讯故障";
        break;
    case sdk_err.ERR_NO_BATTLE_AREA:
        str = "无战斗时间区域";
        break;
    case sdk_err.ERR_NO_TIMER_AREA:
        str = "无秒表区域";
        break;
    case sdk_err.ERR_FPGA_COMM:
        str = "与后级扫描模块通讯故障";
        break;
    case sdk_err.ERR_SET_MODBUS_PARA:
        str = "设置 MODBUS 参数错误";
        break;
    default:
        str = "未知错误: " + err;
        break;
    }
    return str;
}
```